RAPPORT DE TP

Déploiement Automatisé d'une Infrastructure Cloud avec Terraform sur Azure

Introduction

Ce TP a pour objectif de déployer automatiquement une infrastructure cloud complète sur Microsoft Azure en utilisant Terraform. L'infrastructure comprend une machine virtuelle Ubuntu, un stockage Blob Azure ainsi qu'une application web Flask exposée sur internet.

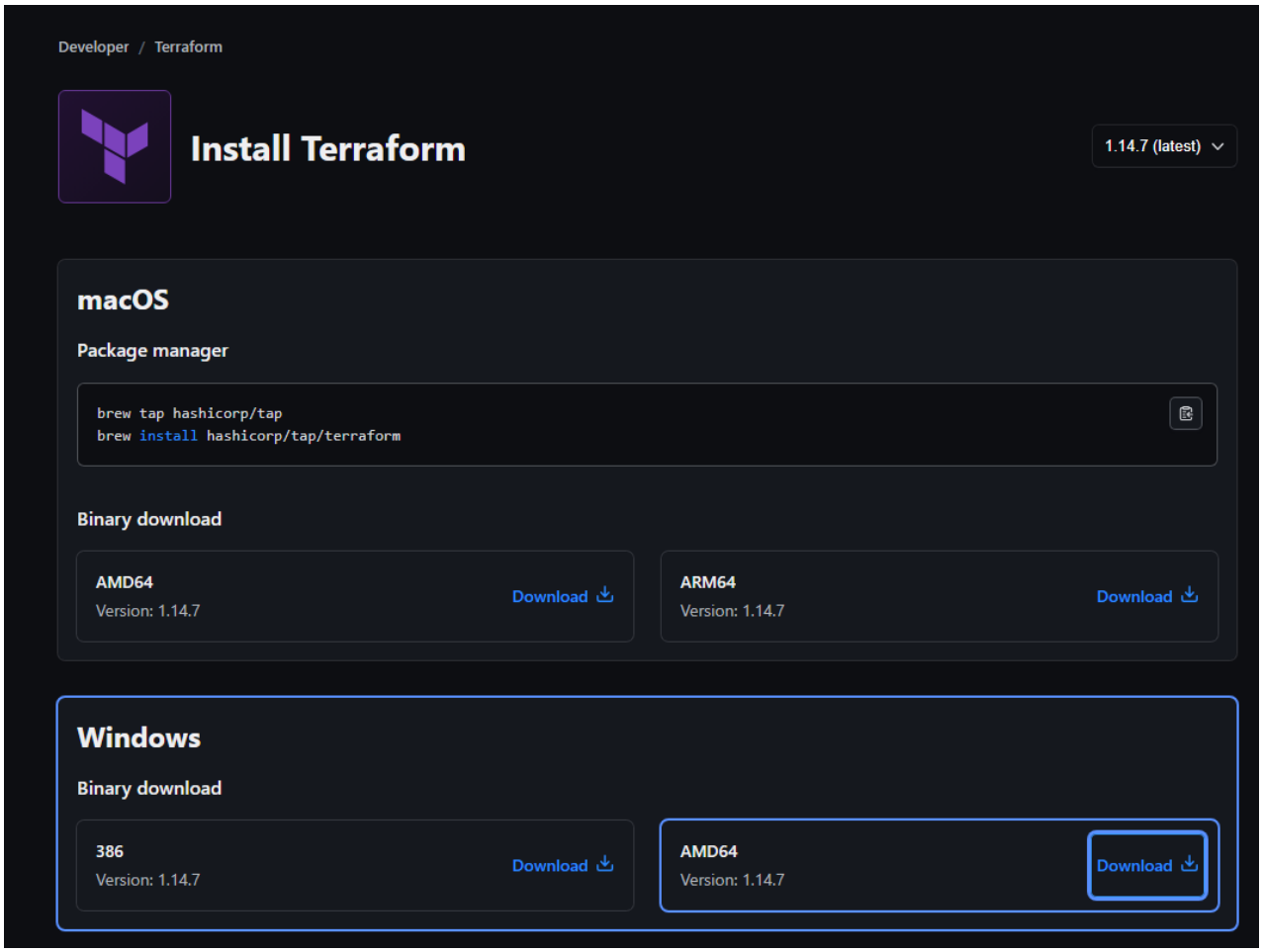
L'ensemble du processus est automatisé : aucune ressource n'est créée manuellement via le portail Azure. Terraform génère, configure et détruit l'infrastructure de façon reproductible.

Étape 1 — Installation de Terraform

La première étape consiste à installer Terraform sur la machine locale (Windows) et à configurer les variables d'environnement pour le rendre accessible depuis le terminal.

1.1 Téléchargement et configuration du PATH

Terraform a été téléchargé depuis le site officiel HashiCorp, placé dans C:\terraform\ et ajouté aux variables d'environnement système.




```
C:\Users\adamr\Downloads\terraform>terraform version
Terraform v1.14.7
on windows_amd64
```

Étape 2 — Configuration d'Azure CLI et authentification

Azure CLI permet de s'authentifier auprès de Microsoft Azure depuis le terminal. Terraform utilise cette session pour créer les ressources cloud.

2.1 Connexion au compte Azure

La commande az login ouvre le navigateur et permet de se connecter avec le compte Azure for Students. Une fois connecté, l'abonnement actif est affiché dans le terminal.

```
C:\Users\adamr>az login
Select the account you want to log in with. For more information on login with Azure CLI, see https://go.microsoft.com/fwlink/?linkid=2271136

Retrieving tenants and subscriptions for the selection...

[Tenant and subscription selection]

No     Subscription name    Subscription ID                       Tenant
-----  -------------------  ------------------------------------  --------
[1] *  Azure for Students   6b5abeb1-2cdc-423e-a5f2-2e690d928b24  Efrei

The default is marked with an *; the default tenant is 'Efrei' and subscription is 'Azure for Students' (6b5abeb1-2cdc-423e-a5f2-2e690d928b24).

Select a subscription and tenant (Type a number or Enter for no changes):

Tenant: Efrei
Subscription: Azure for Students (6b5abeb1-2cdc-423e-a5f2-2e690d928b24)

[Announcements]
With the new Azure CLI login experience, you can select the subscription you want to use more easily. Learn more about it and its configuration at https://go.microsoft.com/fwlink/?linkid=2271236

If you encounter any problem, please open an issue at https://aka.ms/azclibug

[Warning] The login output has been updated. Please be aware that it no longer displays the full list of available subscriptions by default.
```

```
C:\tp-azure>dir C:\tp-azure
 Le volume dans le lecteur C n'a pas de nom.
 Le numéro de série du volume est C41C-78FD

 Répertoire de C:\tp-azure

12/03/2026  14:57    <DIR>          .
12/03/2026  14:57    <DIR>          ..
12/03/2026  14:57               318 provider.tf
               1 fichier(s)              318 octets
               2 Rép(s)  44 695 998 464 octets libres
```

Étape 3 — Création des fichiers Terraform

L'infrastructure est décrite dans 4 fichiers Terraform : provider.tf (configuration du provider Azure), variables.tf (paramètres), main.tf (ressources) et outputs.tf (sorties).

3.1 Structure du projet

```
C:\tp-azure>notepad variables.tf

C:\tp-azure>notepad outputs.tf

C:\tp-azure>dir C:\tp-azure
 Le volume dans le lecteur C n'a pas de nom.
 Le numéro de série du volume est C41C-78FD

 Répertoire de C:\tp-azure

12/03/2026  14:58    <DIR>          .
12/03/2026  14:58    <DIR>          ..
12/03/2026  14:58               247 outputs.tf
12/03/2026  14:57               318 provider.tf
12/03/2026  14:58               337 variables.tf
               3 fichier(s)              902 octets
               2 Rép(s)  44 693 704 704 octets libres

C:\tp-azure>notepad main.tf

C:\tp-azure>dir C:\tp-azure
 Le volume dans le lecteur C n'a pas de nom.
 Le numéro de série du volume est C41C-78FD

 Répertoire de C:\tp-azure

12/03/2026  15:00    <DIR>          .
12/03/2026  15:00    <DIR>          ..
12/03/2026  15:00             4 339 main.tf
12/03/2026  14:58               247 outputs.tf
12/03/2026  14:57               318 provider.tf
12/03/2026  14:58               337 variables.tf
               4 fichier(s)            5 241 octets
               2 Rép(s)  44 695 711 744 octets libres
```

3.2 Initialisation de Terraform

La commande terraform init télécharge automatiquement le provider AzureRM depuis le registre HashiCorp.

```
C:\tp-azure>terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/random versions matching "~> 3.5"...
- Finding hashicorp/azurerm versions matching "~> 3.75"...
- Installing hashicorp/random v3.8.1...
- Installed hashicorp/random v3.8.1 (signed by HashiCorp)
- Installing hashicorp/azurerm v3.117.1...
- Installed hashicorp/azurerm v3.117.1 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Étape 4 — Déploiement de l'infrastructure

La commande terraform apply crée toutes les ressources définies dans main.tf : groupe de ressources, réseau virtuel, sous-réseau, IP publique, NSG, interface réseau, VM Ubuntu et Storage Account avec ses containers.

4.1 Plan de déploiement

Avant d'appliquer, terraform plan liste les 11 ressources à créer et permet de vérifier qu'aucune modification involontaire ne sera effectuée.

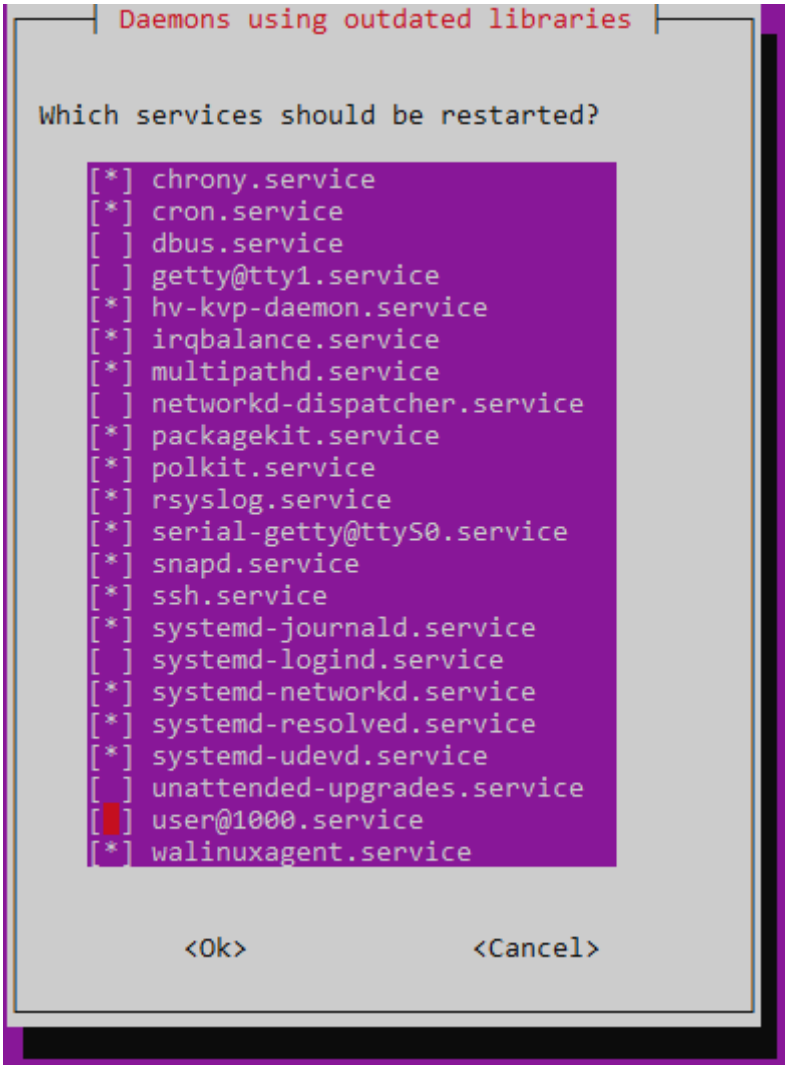
```
C:\tp-azure>terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # azurerm_linux_virtual_machine.main will be created
  + resource "azurerm_linux_virtual_machine" "main" {
      + admin_password                                       = (sensitive value)
      + admin_username                                       = "azureuser"
      + allow_extension_operations                           = true
      + bypass_platform_safety_checks_on_user_schedule_enabled = false
      + computer_name                                        = (known after apply)
      + disable_password_authentication                      = false
      + disk_controller_type                                 = (known after apply)
      + extensions_time_budget                               = "PT1H30M"
      + id                                                   = (known after apply)
      + location                                             = "westeurope"
      + max_bid_price                                        = -1
      + name                                                 = "vm-flask-devops"
      + network_interface_ids                                = (known after apply)
      + patch_assessment_mode                                = "ImageDefault"
      + patch_mode                                           = "ImageDefault"
      + platform_fault_domain                                = -1
      + priority                                             = "Regular"
      + private_ip_address                                   = (known after apply)
      + private_ip_addresses                                 = (known after apply)
      + provision_vm_agent                                   = true
      + public_ip_address                                    = (known after apply)
      + public_ip_addresses                                  = (known after apply)
      + resource_group_name                                  = "rg-flask-devops"
      + size                                                 = "Standard_B1s"
      + virtual_machine_id                                   = (known after apply)
      + vm_agent_platform_updates_enabled                    = false

      + os_disk {
          + caching                   = "ReadWrite"
          + disk_size_gb              = (known after apply)
          + name                      = (known after apply)
          + storage_account_type      = "Standard_LRS"
          + write_accelerator_enabled = false
        }

      + source_image_reference {
          + offer     = "0001-com-ubuntu-server-jammy"
          + publisher = "Canonical"
          + sku       = "22_04-lts-gen2"
          + version   = "latest"
        }

      + termination_notification (known after apply)
    }

  # azurerm_network_interface.main will be created
```

```
  # azurerm_virtual_network.main will be created
  + resource "azurerm_virtual_network" "main" {
      + address_space       = [
          + "10.0.0.0/16",
        ]
      + dns_servers         = (known after apply)
      + guid                = (known after apply)
      + id                  = (known after apply)
      + location            = "westeurope"
      + name                = "vnet-flask-devops"
      + resource_group_name = "rg-flask-devops"
      + subnet              = (known after apply)
    }

Plan: 11 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + flask_app_url        = (known after apply)
  + storage_account_name = "stflaskdevopsdev"
  + vm_public_ip         = (known after apply)

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
```

4.2 Application et création des ressources

Après confirmation, Terraform crée toutes les ressources dans l'ordre correct en respectant les dépendances entre elles.

```
C:\tp-azure>ssh azureuser@20.216.161.66
The authenticity of host '20.216.161.66 (20.216.161.66)' can't be established.
ED25519 key fingerprint is SHA256:u1rCk26vr1Q8hQb1sC3elEBn4pIfGMwosVsXTg/CLKA.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '20.216.161.66' (ED25519) to the list of known hosts.
azureuser@20.216.161.66's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1044-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Thu Mar 12 16:22:48 UTC 2026

  System load:  0.05              Processes:             131
  Usage of /:   5.5% of 28.89GB   Users logged in:       0
  Memory usage: 8%                IPv4 address for eth0: 10.0.1.4
  Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
```



Les outputs affichent l'IP publique de la VM (20.216.161.66), l'URL Flask et le nom du Storage Account (stflasktp2024).

Étape 5 — Connexion à la VM et déploiement Flask

Une fois la VM créée, on se connecte en SSH et on installe Python 3, pip ainsi que la bibliothèque Flask.

5.1 Connexion SSH

```
azureuser@vm-flask-devops:~$ sudo apt update && sudo apt install -y python3 python3-pip
Hit:1 http://azure.archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://azure.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:3 http://azure.archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:4 http://azure.archive.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:5 http://azure.archive.ubuntu.com/ubuntu jammy/universe amd64 Packages [14.1 MB]
Get:6 http://azure.archive.ubuntu.com/ubuntu jammy/universe Translation-en [5652 kB]
Get:7 http://azure.archive.ubuntu.com/ubuntu jammy/universe amd64 c-n-f Metadata [286 kB]
Get:8 http://azure.archive.ubuntu.com/ubuntu jammy/multiverse amd64 Packages [217 kB]
Get:9 http://azure.archive.ubuntu.com/ubuntu jammy/multiverse Translation-en [112 kB]
Get:10 http://azure.archive.ubuntu.com/ubuntu jammy/multiverse amd64 c-n-f Metadata [8372 B]
Get:11 http://azure.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3286 kB]
Get:12 http://azure.archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [499 kB]
Get:13 http://azure.archive.ubuntu.com/ubuntu jammy-updates/main amd64 c-n-f Metadata [19.2 kB]
Get:14 http://azure.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [5414 kB]
Get:15 http://azure.archive.ubuntu.com/ubuntu jammy-updates/restricted Translation-en [1028 kB]
Get:16 http://azure.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 c-n-f Metadata [676 B]
Get:17 http://azure.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1257 kB]
Get:18 http://azure.archive.ubuntu.com/ubuntu jammy-updates/universe Translation-en [315 kB]
Get:19 http://azure.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 c-n-f Metadata [30.4 kB]
Get:20 http://azure.archive.ubuntu.com/ubuntu jammy-updates/multiverse amd64 Packages [59.0 kB]
Get:21 http://azure.archive.ubuntu.com/ubuntu jammy-updates/multiverse Translation-en [13.5 kB]
Get:22 http://azure.archive.ubuntu.com/ubuntu jammy-updates/multiverse amd64 c-n-f Metadata [612 B]
Get:23 http://azure.archive.ubuntu.com/ubuntu jammy-backports/main amd64 Packages [69.4 kB]
Get:24 http://azure.archive.ubuntu.com/ubuntu jammy-backports/main Translation-en [11.5 kB]
Get:25 http://azure.archive.ubuntu.com/ubuntu jammy-backports/main amd64 c-n-f Metadata [412 B]
Get:26 http://azure.archive.ubuntu.com/ubuntu jammy-backports/restricted amd64 c-n-f Metadata [116 B]
Get:27 http://azure.archive.ubuntu.com/ubuntu jammy-backports/universe amd64 Packages [31.7 kB]
```

5.2 Installation de Flask

```
azureuser@vm-flask-devops:~$ pip install flask azure-storage-blob
Defaulting to user installation because normal site-packages is not writeable
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl (103 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 103.4/103.4 KB 3.2 MB/s eta 0:00:00
Collecting azure-storage-blob
  Downloading azure_storage_blob-12.28.0-py3-none-any.whl (431 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 431.5/431.5 KB 10.6 MB/s eta 0:00:00
Collecting markupsafe>=2.1.1
  Downloading markupsafe-3.0.3-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (20 kB)
Collecting itsdangerous>=2.2.0
  Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting click>=8.1.3
  Downloading click-8.3.1-py3-none-any.whl (108 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 108.3/108.3 KB 10.1 MB/s eta 0:00:00
Collecting jinja2>=3.1.2
  Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 134.9/134.9 KB 8.0 MB/s eta 0:00:00
Collecting blinker>=1.9.0
  Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting werkzeug>=3.1.0
  Downloading werkzeug-3.1.6-py3-none-any.whl (225 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 225.2/225.2 KB 9.8 MB/s eta 0:00:00
Collecting azure-core>=1.30.0
  Downloading azure_core-1.38.2-py3-none-any.whl (217 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 218.0/218.0 KB 6.5 MB/s eta 0:00:00
Requirement already satisfied: cryptography>=2.1.4 in /usr/lib/python3/dist-packages (from azure-storage-blob) (3.4.8)
Collecting isodate>=0.6.1
  Downloading isodate-0.7.2-py3-none-any.whl (22 kB)
Collecting typing-extensions>=4.6.0
  Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.6/44.6 KB 1.1 MB/s eta 0:00:00
Requirement already satisfied: requests>=2.21.0 in /usr/lib/python3/dist-packages (from azure-core>=1.30.0->azure-storage-blob) (2.25.1)
Installing collected packages: typing-extensions, markupsafe, itsdangerous, isodate, click, blinker, werkzeug, jinja2, azure-core, flask, azure-storage-blob
  WARNING: The script flask is installed in '/home/azureuser/.local/bin' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed azure-core-1.38.2 azure-storage-blob-12.28.0 blinker-1.9.0 click-8.3.1 flask-3.1.3 isodate-0.7.2 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 typing-extensions-4.15.0 werkzeug-3.1.6
```

5.3 Lancement de l'application

L'application Flask est créée avec 3 endpoints : GET /, GET /files et POST /files. Elle est lancée sur le port 5000 et écoute sur toutes les interfaces réseau (0.0.0.0).

```
azureuser@vm-flask-devops:~/flask-app$ nano app.py
azureuser@vm-flask-devops:~/flask-app$ _
```

Étape 6 — Tests de l'application

L'application Flask est testée depuis un navigateur web en utilisant l'IP publique de la VM.

6.1 Test de l'endpoint principal

```python
from flask import Flask, jsonify, request

app = Flask(__name__)

@app.route('/')
def home():
    return jsonify({"message": "Flask API is running!", "status": "ok"})

@app.route('/files', methods=['GET'])
def list_files():
    files = ["image1.png", "log1.txt"]
    return jsonify({"files": files})

@app.route('/files', methods=['POST'])
def upload_file():
    data = request.json
    return jsonify({"message": "File uploaded", "file": data.get("name")}), 201

@app.route('/files/<filename>', methods=['DELETE'])
def delete_file(filename):
    return jsonify({"message": f"File {filename} deleted"})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

6.2 Test de l'endpoint /files

```
azureuser@vm-flask-devops:~/flask-app$ python3 app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.0.1.4:5000
Press CTRL+C to quit
```

Étape 7 — Destruction de l'infrastructure

La commande terraform destroy supprime proprement toutes les ressources créées sur Azure. C'est une étape importante pour éviter des coûts inutiles et démontrer la reproductibilité de l'infrastructure.

Non sécurisé 20.216.161.66:5000

Impression élégante

{"message":"Flask API is running!","status":"ok"}

```
Plan: 0 to add, 0 to change, 11 to destroy.

Changes to Outputs:
  - flask_app_url        = "http://20.216.161.66:5000" -> null
  - storage_account_name = "stflasktp2024" -> null
  - vm_public_ip         = "20.216.161.66" -> null
azurerm_network_interface_security_group_association.main: Destroying... [id=/subsc
/6b5abeb1-2cdc-423e-a5f2-2e690d928b24/resourceGroups/rg-flask-devops/providers/Micr
azurerm_storage_container.images: Destroying... [id=https://stflasktp2024.blob.core
azurerm_storage_container.logs: Destroying... [id=https://stflasktp2024.blob.core.w
azurerm_linux_virtual_machine.main: Destroying... [id=/subscriptions/6b5abeb1-2cdc-
azurerm_network_interface_security_group_association.main: Still destroying... [id=
azurerm_storage_container.images: Still destroying... [id=https://stflasktp2024.blo
azurerm_storage_container.logs: Still destroying... [id=https://stflasktp2024.blob.
azurerm_linux_virtual_machine.main: Still destroying... [id=/subscriptions/6b5abeb1
azurerm_network_interface_security_group_association.main: Destruction complete aft
azurerm_network_security_group.main: Destroying... [id=/subscriptions/6b5abeb1-2cdc
azurerm_storage_container.images: Destruction complete after 16s
azurerm_storage_container.logs: Destruction complete after 19s
azurerm_storage_account.main: Destroying... [id=/subscriptions/6b5abeb1-2cdc-423e-a
azurerm_linux_virtual_machine.main: Still destroying... [id=/subscriptions/6b5abeb1
azurerm_network_security_group.main: Still destroying... [id=/subscriptions/6b5abeb
azurerm_storage_account.main: Destruction complete after 10s
azurerm_linux_virtual_machine.main: Still destroying... [id=/subscriptions/6b5abeb1
azurerm_network_security_group.main: Destruction complete after 18s
azurerm_linux_virtual_machine.main: Still destroying... [id=/subscriptions/6b5abeb1
azurerm_linux_virtual_machine.main: Still destroying... [id=/subscriptions/6b5abeb1
azurerm_linux_virtual_machine.main: Destruction complete after 56s
azurerm_network_interface.main: Destroying... [id=/subscriptions/6b5abeb1-2cdc-423e
azurerm_network_interface.main: Still destroying... [id=/subscriptions/6b5abeb1-2cd
azurerm_network_interface.main: Destruction complete after 17s
azurerm_public_ip.main: Destroying... [id=/subscriptions/6b5abeb1-2cdc-423e-a5f2-2e
azurerm_subnet.main: Destroying... [id=/subscriptions/6b5abeb1-2cdc-423e-a5f2-2e690
azurerm_public_ip.main: Still destroying... [id=/subscriptions/6b5abeb1-2cdc-423e-a
azurerm_subnet.main: Still destroying... [id=/subscriptions/6b5abeb1-2cdc-423e-a5f2
azurerm_public_ip.main: Destruction complete after 15s
azurerm_subnet.main: Destruction complete after 18s
azurerm_virtual_network.main: Destroying... [id=/subscriptions/6b5abeb1-2cdc-423e-a
azurerm_virtual_network.main: Still destroying... [id=/subscriptions/6b5abeb1-2cdc-
azurerm_virtual_network.main: Destruction complete after 15s
azurerm_resource_group.main: Destroying... [id=/subscriptions/6b5abeb1-2cdc-423e-a5
azurerm_resource_group.main: Still destroying... [id=/subscriptions/6b5abeb1-2cdc-4
azurerm_resource_group.main: Destruction complete after 19s

Destroy complete! Resources: 11 destroyed.
```

Conclusion

Ce TP a permis de mettre en place une infrastructure cloud complète et automatisée sur Microsoft Azure en utilisant Terraform. L'ensemble du cycle de vie de l'infrastructure a été couvert :

- Création automatique de la VM Ubuntu, du réseau et du stockage avec terraform apply
- Déploiement d'une application Flask accessible depuis internet
- Suppression propre de toutes les ressources avec terraform destroy

La principale difficulté rencontrée a été les restrictions de l'abonnement Azure for Students qui limitent les régions et les tailles de VM disponibles. Ces problèmes ont été résolus en identifiant les régions autorisées via az policy et en testant différentes tailles de VM jusqu'à trouver Standard_B2ls_v2 en zone 3 de France Central.

L'infrastructure as code avec Terraform présente un avantage majeur : la reproductibilité. En relançant terraform apply, on obtient exactement la même infrastructure à tout moment.